Principal Component Analysis Project

Principal Component Analysis on Yale Face Images

Mojdeh Fatehpour and Mohammadmatin Rezanezhoulaei

Università degli Studi della Campania "Luigi Vanvitelli"

Prof. Rosanna Campagna

Abstract:
The goal of this project is to investigate how well the low-rank approximation of the Yale Face Database reproduces facial images using Principal Component Analysis (PCA). We extract representative faces, analyze singular value decay, plot eigenfaces, and perform image reconstruction with varying ranks. Results demonstrate that PCA is effective for dimensionality reduction while preserving essential image features in compressed reconstructions.

1. Yale Face Database Overview

The Yale Face Database contains black-and-white photographs of 15 individuals, each with 11 facial images taken under different expressions and lighting conditions (normal, sad, happy, surprised, sleepy, wink; center, left, right lighting; with and without glasses).

The database includes:

- `fea`: a 165×4096 matrix. Each row corresponds to a flattened 64×64 grayscale face image.
- `gnd`: a 165×1 label vector identifying individuals.

We visualize all 165 face images tiled into a single matrix layout for an initial overview of the dataset.

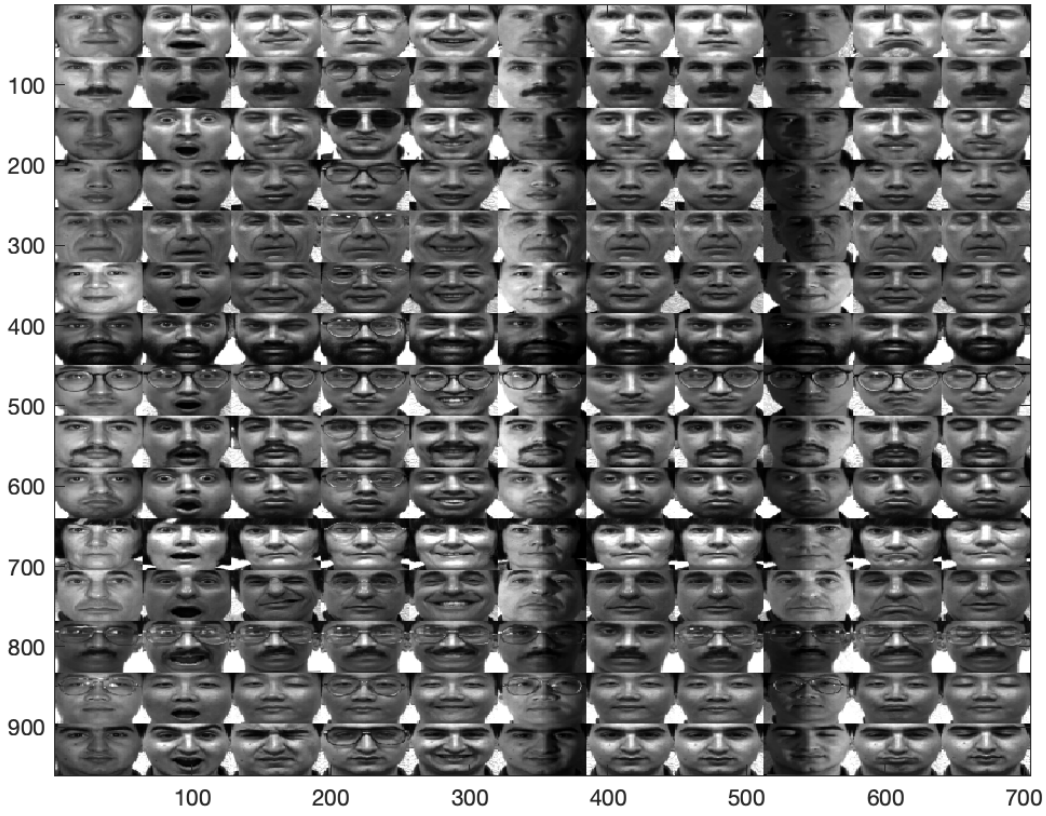

Figure 1. All 165 faces from the Yale Face Database visualized in a matrix layout (64×64 pixels each).

2. Exercise (a): Extract and Visualize a Submatrix of Faces

Problem Description

Extract in a submatrix F and plot 6 different faces for 5 different individuals as gray scale images. Choose faces that you think are in some sense representative between the different groups (normal, sad, happy, surprised, sleepy, wink).

Methodology

To create the submatrix F, we manually selected 6 face images for each of 5 different individuals from the full Yale Face Database, resulting in a total of 30 images. Each image is 64×64 pixels and stored as a row in the matrix `fea`. The selected images represent a variety of facial expressions, including normal, happy, sad, surprised, sleepy, and wink. These images were reshaped from vector form back into their original 2D format and arranged into a 6×5 grid using MATLAB for visualization. This subset F was then used for all subsequent PCA analysis steps.

Result

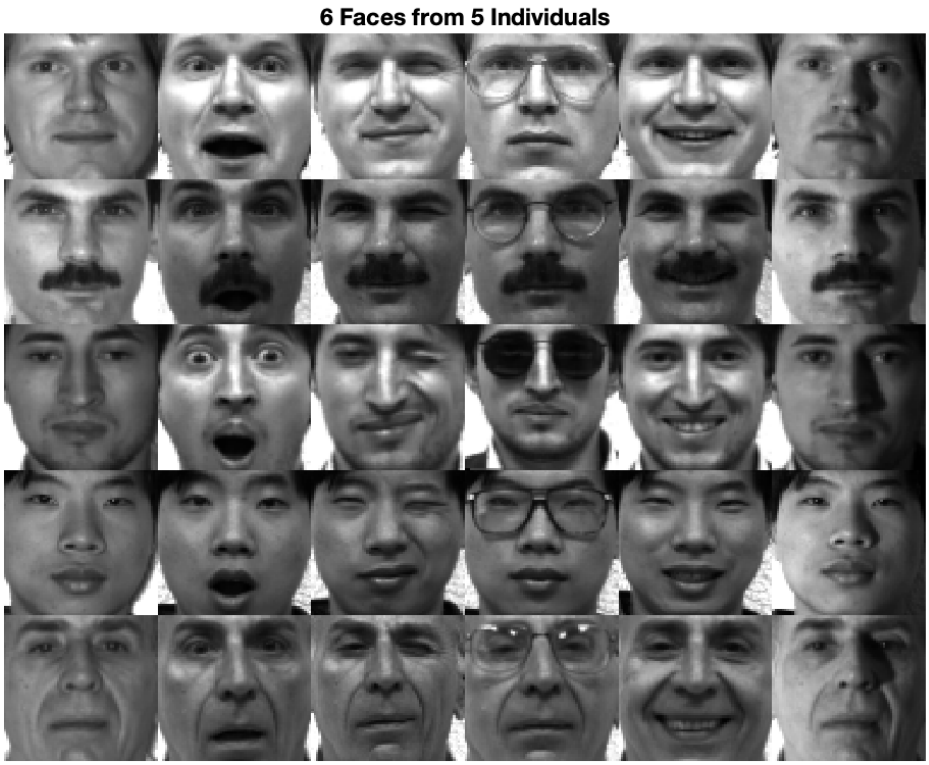

Figure 2. Selected 6×5 grid of faces from the Yale Face Database.

Observation: The resulting grid displays a balanced selection of expressive facial variations across five individuals. Each row corresponds to a single individual showing six distinct facial expressions, capturing variations in mood and eye state (e.g., open/closed, presence of glasses). This diversity is crucial for effective PCA, as it ensures the computed principal components (eigenfaces) capture meaningful variation in facial structure and expression across the dataset.

3. Exercise (b): Singular Value Analysis of F

Problem Description

After having computed the first singular values/vectors of F, plot the singular values and comment on how fast (or slowly) they decrease. Notice, however, that computing the full SVD may be very slow, so use `svds`, increasing gradually r. It may be more informative to plot their logarithms.

Methodology

To examine the distribution of information captured by the principal components, we first centered the matrix F by subtracting the mean face vector from each image (column-wise mean). Then, we applied the sparse SVD function `svds` to compute the first k singular values, using increasing values $k = 10, 20, 30$.

These singular values were plotted in logarithmic scale to better visualize their rate of decay and to interpret how much variance each component retains.

Result

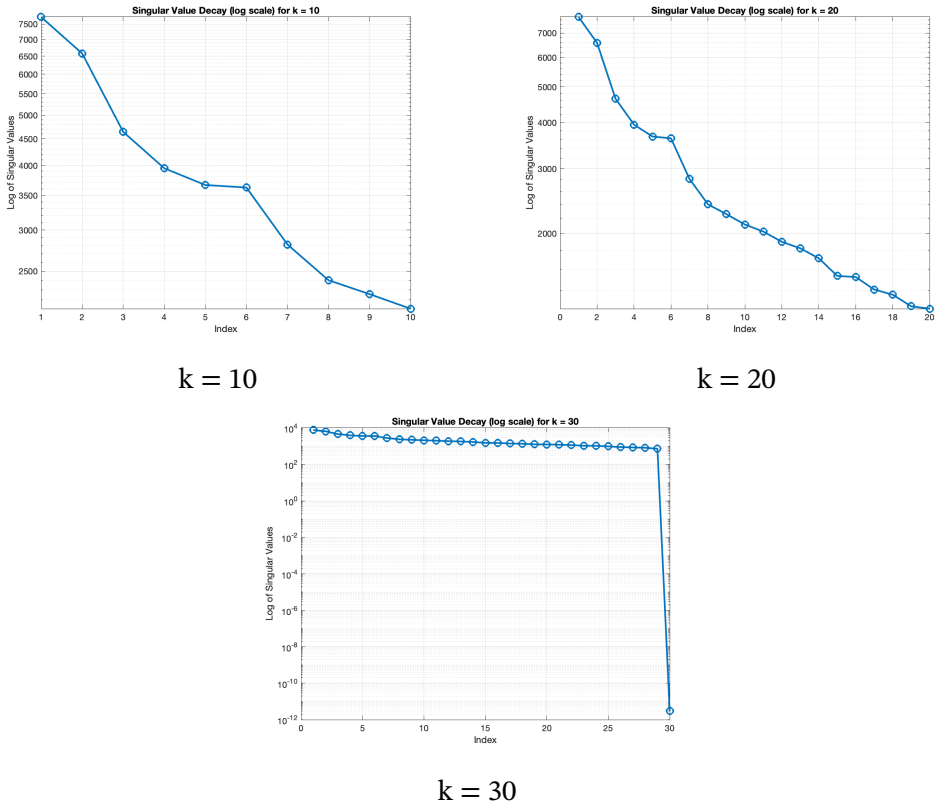

Figure 3. Logarithmic decay of the first k singular values for the centered face matrix F.

Interpretation: For $k = 10$ and $k = 20$, the singular values decrease rapidly, indicating that a large portion of the dataset's variance is captured by the leading components. This supports the idea that facial image data lies on a lower-dimensional manifold. Interestingly, for $k = 30$, the values appear nearly constant on the logarithmic scale, which suggests they are close in magnitude — potentially due to numerical limitations or the fact that higher-order components capture little distinct structure. Overall, these results validate the use of low-rank approximation in this context and highlight PCA's effectiveness in compressing the dataset with minimal information loss.

4. Exercise (c): Plot the First 5 Eigenfaces

Problem Description

Plot the first 5 feature vectors (columns of U) as gray scale images.

Methodology

After computing the SVD of the centered face matrix F, the first 5 columns of the left singular matrix U were selected. Each column vector represents a principal component (PC), capturing a direction of

maximum variance in the image space. These vectors were reshaped from 4096-dimensional vectors into 64×64 grayscale images and visualized as "eigenfaces." The resulting plots show the key components that PCA uses to reconstruct facial images.

Result

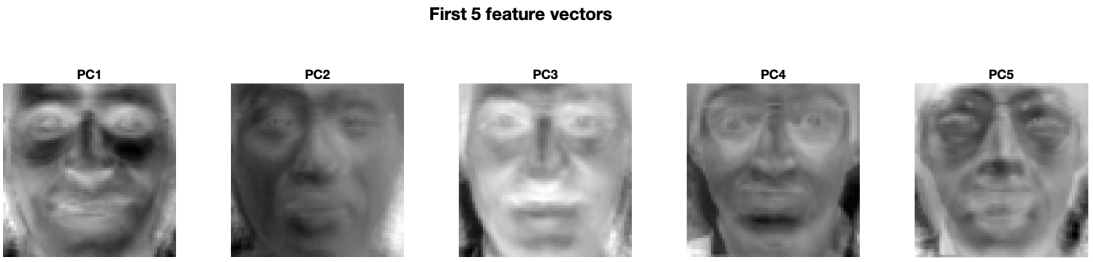

Figure 4. Top 5 eigenfaces (principal components) extracted from the centered dataset F.

Observation: These eigenfaces reveal dominant modes of variation across the dataset. For example, PC1 emphasizes overall brightness and face shape, while subsequent PCs highlight features such as eye sockets, mouth, nose, and lighting direction. Some components resemble shadows or lighting gradients, which means PCA is capturing not only anatomical differences but also expression and illumination changes. These components serve as a low-dimensional basis from which facial images can be accurately reconstructed.

5. Exercise (d): Image Reconstruction with Varying Rank

Problem Description

Approximate the 5 images corresponding to the columns that you selected by a linear combination of the first $k = 4; 8; 15$ feature vectors with coefficients their principal components. Each time display the approximation and difference between it and the original data in the form of a gray scale image.

Methodology

We selected 5 original face images and projected them onto the subspace spanned by the first k eigenfaces, for $k = 4, 8, 15$. Each reconstruction was computed by multiplying the eigenface basis with the corresponding principal component coefficients for that image, then adding back the mean face to restore pixel intensities. For each value of k, we visualized:

- The original face image (top row),
- The reconstructed image (middle row),
- The residual (difference between original and reconstruction, bottom row).

These comparisons help evaluate how well each low-rank approximation preserves facial structure.

Result

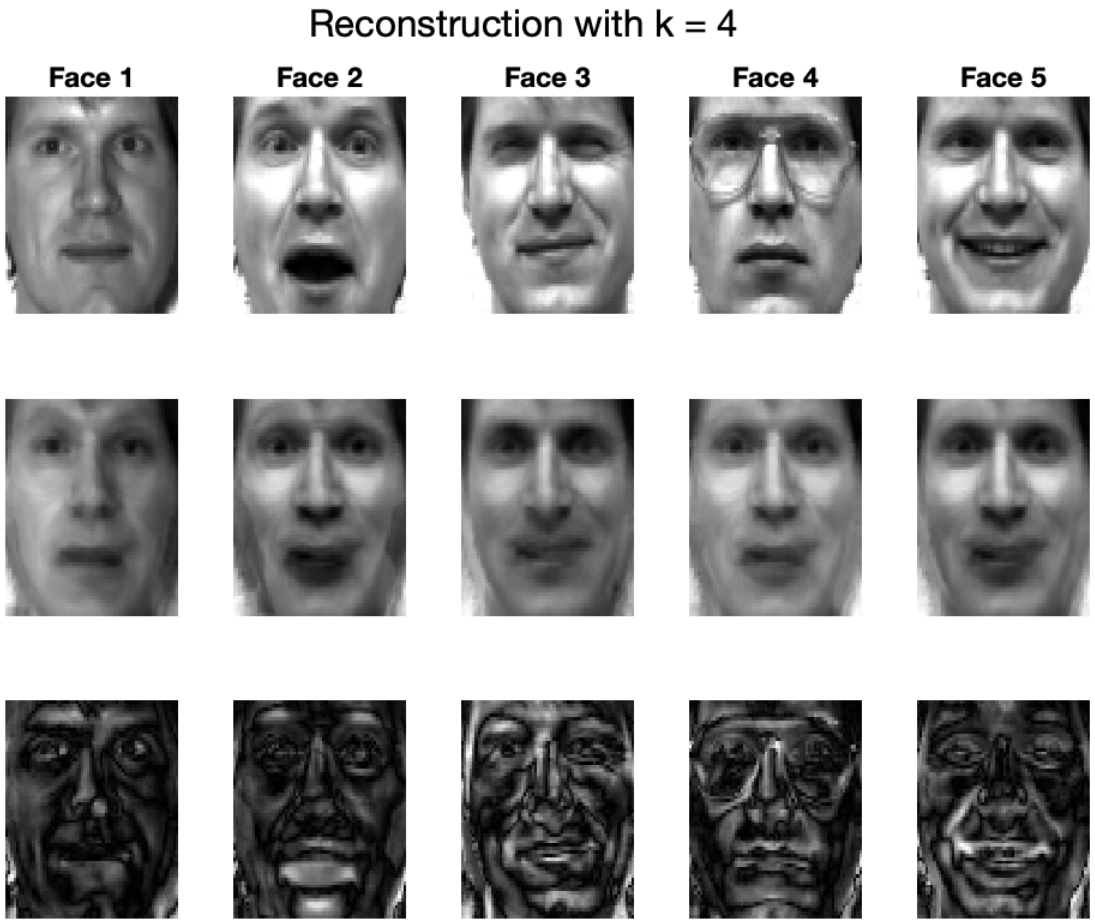

Figure 5. Reconstruction with $k = 4$: original, approximation, and residuals.

Interpretation (k = 4): With only 4 components, the general facial layout is preserved, but fine details such as glasses, mouth shape, and eye texture are lost. Residuals show sharp edges and areas of high contrast, indicating significant reconstruction error.

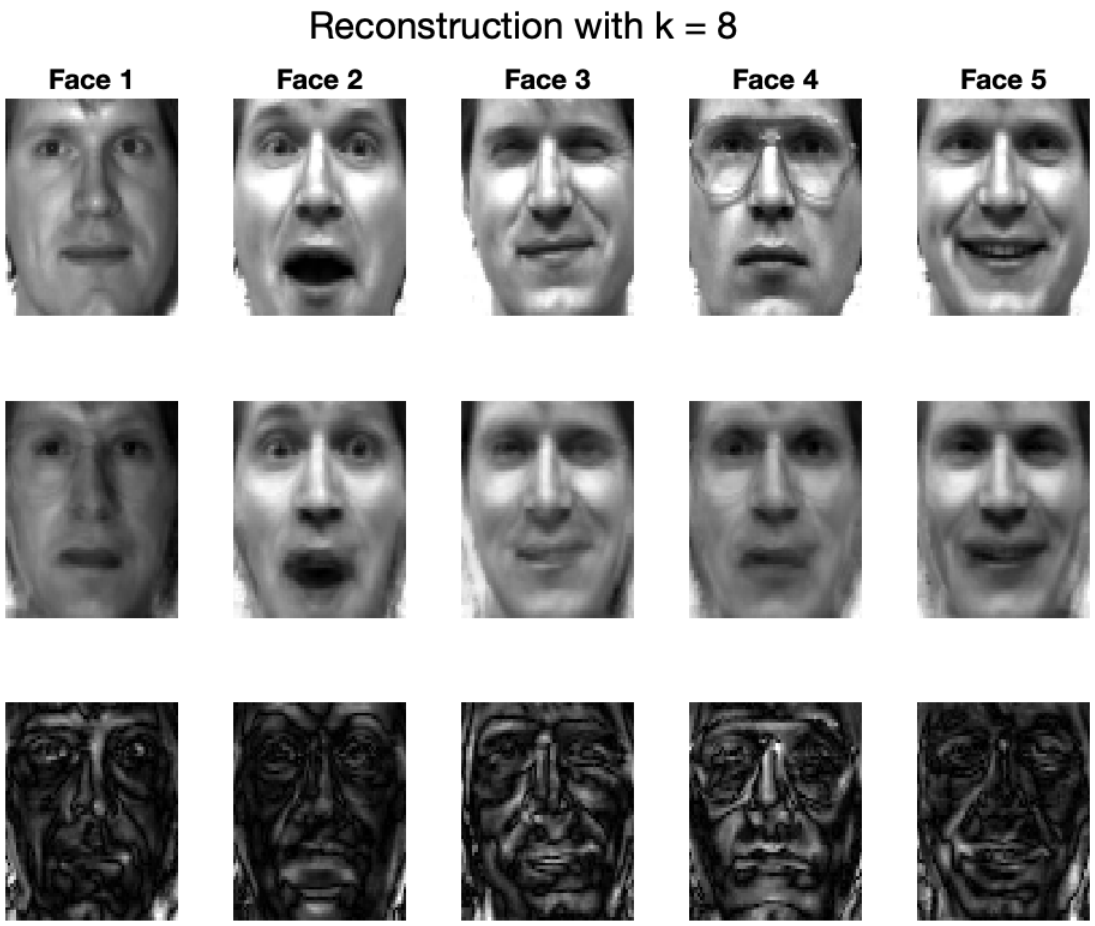

Figure 6. Reconstruction with $k = 8$: original, approximation, and residuals.

Interpretation (k = 8): At 8 components, reconstructions capture more facial detail and expressions more clearly. Glasses and eye regions are more distinct. Residuals still show some information loss, but with reduced intensity and sharper approximation quality.

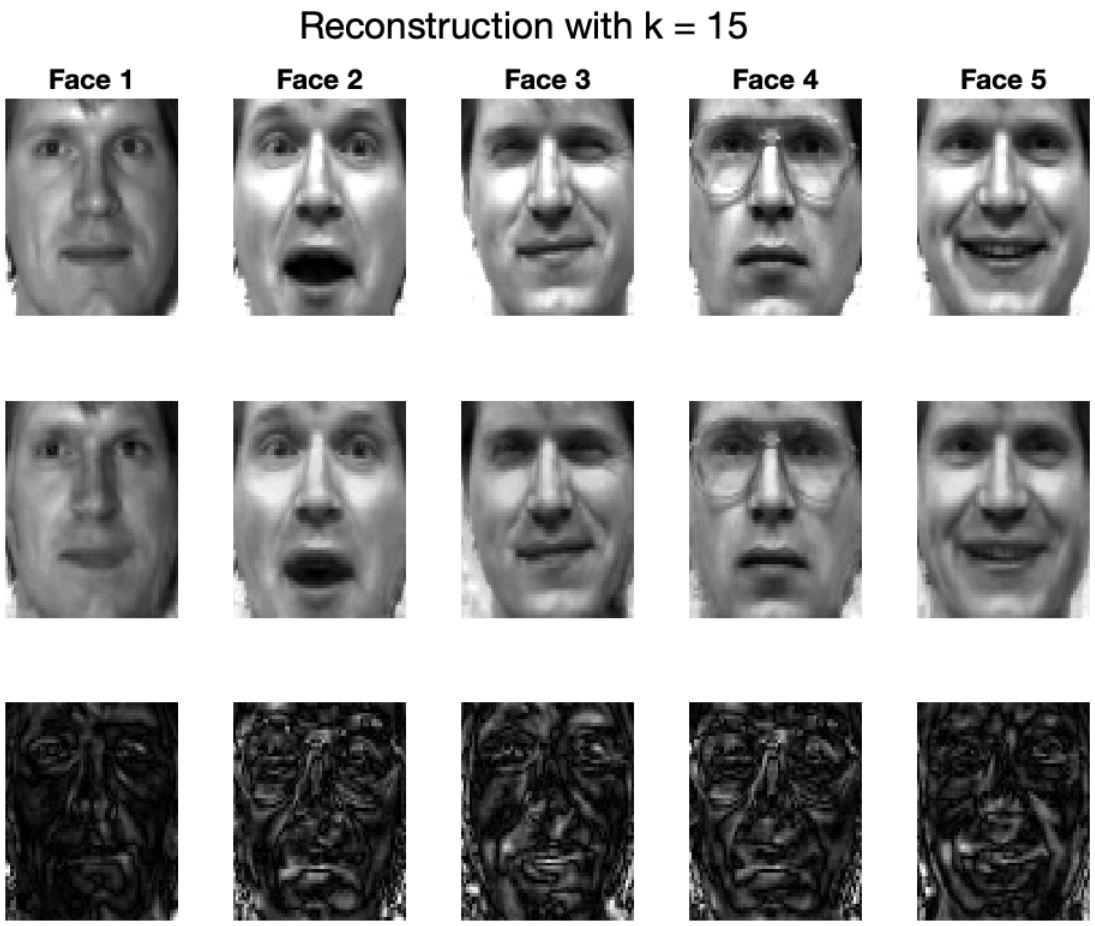

Figure 7. Reconstruction with $k = 15$: original, approximation, and residuals.

Interpretation (k = 15): Using 15 components produces reconstructions that are nearly identical to the originals. Facial features, expressions, and lighting are all preserved. Residual images are faint, indicating that very little information has been lost in the low-rank approximation. This confirms that a relatively small number of eigenfaces can represent high-dimensional facial data effectively.

6. Conclusion

This project set out to evaluate how well low-rank approximations via Principal Component Analysis (PCA) can reconstruct facial images while significantly reducing data dimensionality. Through a series of structured experiments, we confirmed that PCA captures the most relevant facial information in just a few principal components.

The selected subset of faces provided sufficient variation to expose meaningful structure. The decay of singular values revealed that most variance is concentrated in the first few dimensions. Eigenfaces visualized the dominant spatial features learned by PCA, while reconstruction results showed that using only 15 components yielded nearly lossless recovery of the original faces.

Importantly, residuals between the original and reconstructed images decreased significantly with

increasing rank, validating the effectiveness of PCA as a low-rank approximation tool. While performance at very low k values was visibly imperfect, the method still preserved core identity and expression.

Overall, the exercises demonstrate that PCA is not only theoretically sound but also practically effective for grayscale face image compression, offering a balance between dimensionality reduction and reconstruction quality.

Appendix: MATLAB Code

```
%==========================================

close all
clear
%load Yale_64x64.mat
load Yale_64x64.mat
faceW = 64;
faceH = 64;
numPerLine = 11;
ShowLine = 15;
Y = zeros(faceH*ShowLine,faceW*numPerLine);
for i=0:ShowLine-1
for j=0:numPerLine-1
Y(i*faceH+1:(i+1)*faceH,j*faceW+1:(j+1)*faceW) = reshape(fea(i*numPerLine+j+1,:),[faceH,
    ↪ faceW]);
end
end
imagesc(Y);colormap(gray);
saveas(gcf, 'faces.png')

%==========================================

%(a) Extract in a submatrix F and plot 6 different faces for 5 different individuals as
    ↪ gray scale images.
% Choose faces that you think are in some sense representative between the different
    ↪ groups (normal, sad, happy, surprised, sleepy, wink).
X = fea;
numPerLine = 6; % faces per person
ShowLine = 5;   % number of people

F = zeros(numPerLine * ShowLine, size(X, 2));  % 30 rows (6x5), 4096 columns
```

```
Y = zeros(faceH * ShowLine, faceW * numPerLine);  % for displaying the faces

count = 1;
for i = 0:ShowLine - 1
    for j = 0:numPerLine - 1
        index = i * 11 + j + 1;

        % Display face in the big image
        Y(i*faceH+1:(i+1)*faceH, j*faceW+1:(j+1)*faceW) = ...
            reshape(X(index, :), [faceH, faceW]);

        % Store face in F
        F(count, :) = X(index, :);

        count = count + 1;
    end
end

figure();
imagesc(Y); colormap(gray); axis off;
title('6 Faces from 5 Individuals');
saveas(gcf, 'faces_6x5.png');

%=========================================

%b) After having computed the first singular values/vectors of F,
% plot the singular values and comment on how fast (or slowly) they decrease.
% Notice, however, that computing the full SVD may be very slow, so use svds,
% increasing gradually r. It may be more informative to plot their logarithms.

% PCA Singular Value Decay  Three Separate Plots for k = 10, 20, 30

% Step 1: Center the data
mean_face = mean(F', 2);
centering = F' - mean_face;

% Step 2: Define k values for separate plots
k_values = [10, 20, 30];

for i = 1:length(k_values)
```

```
    k = k_values(i);

    % Compute top-k singular values
    [~, D, ~] = svds(centering, k);
    s = diag(D);  % Singular values

    % Create figure
    figure();
    semilogy(1:k, s, '-o', 'LineWidth', 2, 'MarkerSize', 8);
    xlabel('Index');
    ylabel('Log of Singular Values');
    title(['Singular Value Decay (log scale) for k = ', num2str(k)]);
    grid on;

    % Save each plot
    filename = ['singular_values_k', num2str(k), '.png'];
    saveas(gcf, filename);
end

%========================================

%(c) Plot the first 5 feature vectors (columns of U) as gray scale images.

mean_face = mean(F', 2);
centering = F' - mean_face;
k = 5;
[U, ~, ~] = svds(centering, k);

figure();
for i = 1:k
    subplot(1, k, i);
    imagesc(reshape(U(:, i), [64, 64]));
    colormap(gray);
    axis image off;
    title(['PC', num2str(i)], 'FontWeight', 'bold');
end

sgtitle('First 5 feature vectors', 'FontWeight', 'bold');

% Save as PNG
```

```
109 set(gcf, 'Position', [100, 100, 1200, 300]);  % Wider figure with fixed height
110 saveas(gcf, 'eigenfaces_top5_clean.png');
111
112 %=========================================
113
114 %(d) Approximate the 5 images corresponding to the columns that you selected by a linear
115 % combination of the first k = 4; 8; 15 feature vectors with coefficients their
    ↪ principal
116 % components. Each time display the approximation and difference between it and the
117 % original data in the form of a gray scale image.
118
119
120 mean_face = mean(F', 2);
121 centering = F' - mean_face;
122
123 faces_to_reconstruct = 1:5;
124 ks = [4, 8, 15];
125
126 for k = ks
127     [U_k, ~, ~] = svds(centering, k);
128
129     figure();
130     sgtitle(['Reconstruction with k = ', num2str(k)]);
131
132     for i = 1:length(faces_to_reconstruct)
133         idx = faces_to_reconstruct(i);
134         original = F(idx, :)';
135         centered = original - mean_face;
136
137         coeffs = U_k' * centered;
138         reconstruction = U_k * coeffs + mean_face;
139         diff = abs(original - reconstruction);
140
141         % Plot original, reconstruction, and difference
142         subplot(3, length(faces_to_reconstruct), i);
143         imagesc(reshape(original, 64, 64)); colormap(gray); axis off;
144         if i == 1, ylabel('Original'); end
145         title(['Face ', num2str(i)]);
146
147         subplot(3, length(faces_to_reconstruct), i + length(faces_to_reconstruct));
```

```
        imagesc(reshape(reconstruction, 64, 64)); colormap(gray); axis off;
        if i == 1, ylabel('Reconstructed'); end

        subplot(3, length(faces_to_reconstruct), i + 2 * length(faces_to_reconstruct));
        imagesc(reshape(diff, 64, 64)); colormap(gray); axis off;
        if i == 1, ylabel('Difference'); end
    end
    saveas(gcf, ['reconstruction_k' num2str(k) '.png']);
end
```

Code 1. PCA-based face image analysis and reconstruction.